



# CHRONOS: Comprehensive Hackathon Defense & Live Demonstration Guide

---

**Track:** PS-10 — Forensics & Incident Response

**Project:** Chronos — Automated Multi-Source Forensic Timeline Parser & Incident Reconstruction Engine

**Team / Author:** BaggaByte / Gurnoor Bagga

---



## Table of Contents

---

1. Executive Summary & Problem Statement
  2. Complete Technology Stack & Architecture
  3. Deep Dive into the 5 Technical Pillars
  4. Hackathon Live Demonstration Script (Step-by-Step)
  5. Tough Questions & Winning Answers for Judges (Q&A Defense)
  6. Key Metrics, Data Flow & Evidence Summary
- 

## 1. Executive Summary & Problem Statement

---

### The Problem

When a major cybersecurity breach occurs (e.g., aerospace intellectual property theft), digital forensics and incident response (DFIR) teams face hundreds of gigabytes of raw, unstructured, and heterogeneous logs across five completely different operating environments:

1. Linux authentication logs ( `auth.log` )
2. Windows Security Event logs ( `EVTX` )
3. Web application access logs (Apache / Nginx)
4. Perimeter firewall syslogs (Cisco ASA)
5. Cloud management audit logs (AWS CloudTrail)

### The Core Challenges

- **Format Heterogeneity:** Each source logs in completely different syntaxes (RFC 3164 syslog, XML/binary EVTX, Combined Log Format, JSON lines).
- **Timestamp & Timezone Chaos:** Servers reside in different timezones ( `America/New_York` , `Europe/London` , `America/Los_Angeles` , `UTC` ), legacy syslog headers lack year declarations and UTC offsets, and unsynchronized machines exhibit clock drift (e.g., +47 seconds).
- **Needle-in-a-Haystack Correlation:** Manually joining an external IP to an internal jump-host, an escalated Windows account, an SSH session, and an S3 `PutObject` call across spreadsheets takes days.
- **Evidence Integrity & Admissibility:** In manual workflows, analysts modify files or lose chain of custody. Without verifiable cryptographic hashing upon first intake, timeline evidence is vulnerable to legal and forensic

challenge.

## The Chronos Solution

Chronos is an automated DFIR engine and interactive visualization suite that:

- Ingests raw multi-format logs in a streaming manner and generates an immutable, SHA-256 tamper-evident manifest before any transformation.
  - Normalizes all timestamps into universal UTC, resolving missing years and inferring timezone offsets while explicitly flagging them for analyst defensibility ( `offset_inferred: true` ).
  - Employs data-driven MITRE ATT&CK technique mapping to categorize adversary actions.
  - Automatically correlates multi-hop attacker narratives across hosts using 4-dimensional entity linking (IP, User, Session, Hostname).
  - Detects statistical anomalies (rolling z-score spike detection for brute-force attacks and adaptive dwell-time gaps).
  - Renders an interactive, zoomable, and pannable timeline dashboard with an ATT&CK Matrix, cross-host lateral movement graph, and one-click court-ready report exports.
- 

## 2. Complete Technology Stack & Architecture

---

### Backend & Analytics Engine (Python 3.10+)

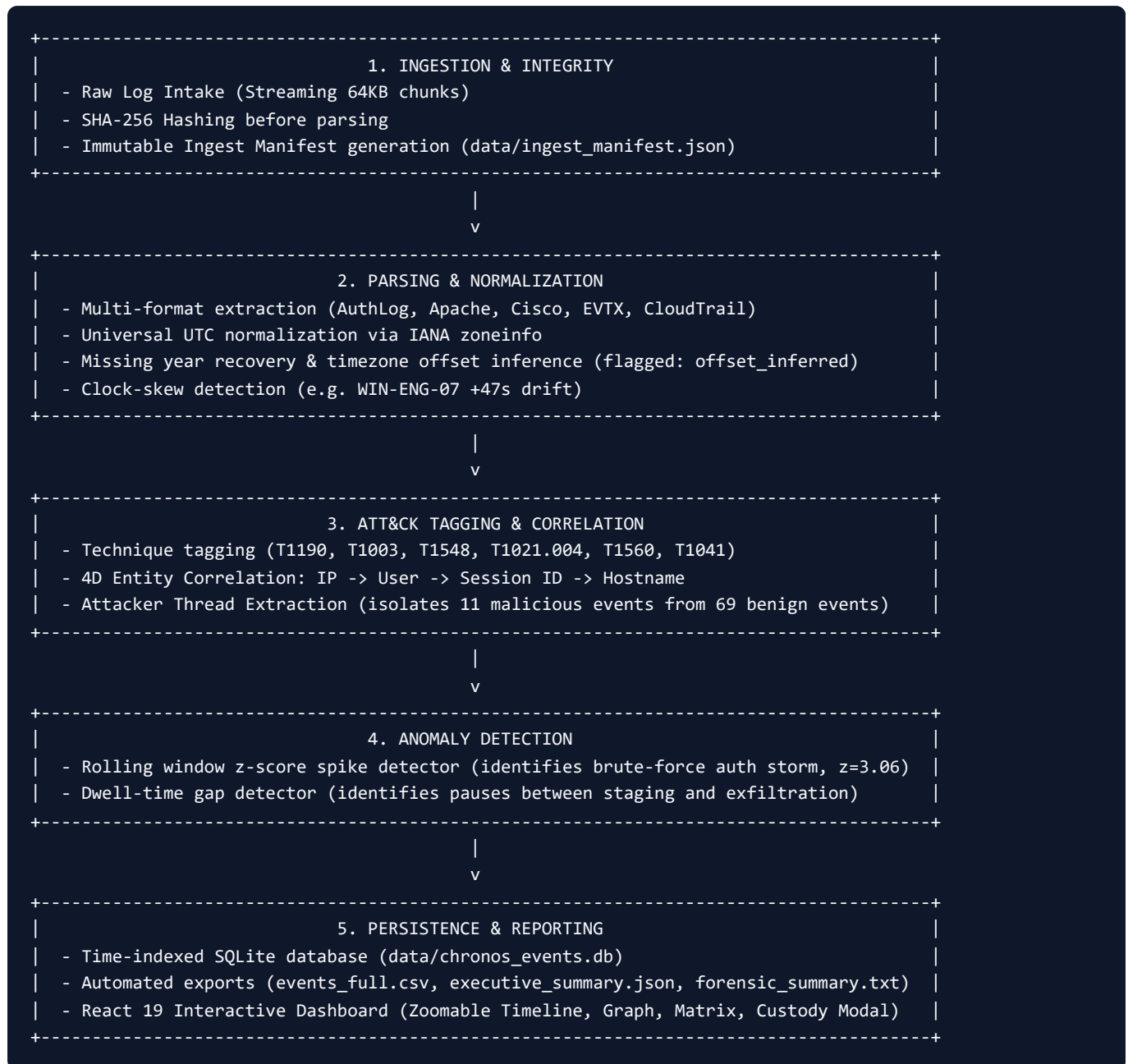
- **Core Language:** Python 3.10+ (utilizing strict typing, `dataclasses`, and type hints).
- **Parsers:** Dedicated streaming parsers for Linux `auth.log`, Apache/Nginx, Cisco Syslog, Windows EVTX/JSON, and AWS CloudTrail.
- **Normalization:** Python `datetime` & standard library `zoneinfo` (IANA timezone database integration) with explicit inference tagging.
- **Evidentiary Security:** `hashlib.sha256` streaming chunk-based hashing ( `src/ingestion/manifest.py` ).
- **Data Persistence:** SQLite ( `sqlite3` ) with time-indexed nano-second precision ( `utc_epoch_ns` ) and an abstract `StorageBackend` interface designed for Elasticsearch/OpenSearch drop-in.
- **Rule Engine:** PyYAML ( `config/attack_mappings.yaml` ) for decoupled MITRE ATT&CK signature matching.
- **Statistical Engine:** NumPy / pure-math rolling z-score spike calculations and median-based dwell-time gap detection.
- **Test Suite:** Standard library `unittest` and `pytest` with 20 comprehensive unit and end-to-end integration tests.

### Frontend Visualizer & Dashboard

- **Framework:** React 19 + TypeScript.
- **Build Tool:** Vite 6.0 with lightning-fast Hot Module Replacement (HMR).
- **Styling:** Vanilla CSS & Tailwind CSS for glassmorphism, responsive cyber-forensic design, and dark mode palette.
- **Visualization Components:** Custom SVG-based zoomable and pannable timeline with non-passive pointer event listeners; SVG topological node-link graph for lateral movement; MITRE ATT&CK tactic heatmaps.
- **Routing:** TanStack Router.

- **Icons & UI:** Lucide React icons with atomic accessible design tokens.

## Architectural Pipeline Flow



## 3. Deep Dive into the 5 Technical Pillars

### Pillar 1: Multi-Format Log Ingestion & Evidentiary Integrity

- **Implementation:** `src/parsers/` and `src/ingestion/manifest.py`.
- **Forensic Significance:** Evidence admitted in court requires strict chain-of-custody documentation. Chronos reads files in read-only mode and computes a SHA-256 hash across 64 KB streaming blocks before passing lines to any regex or JSON parser.

- **Tamper Verification:** The module provides `verify_manifest()`, which recalculates the cryptographic hashes against live disk files at any time to guarantee that logs were never modified.

## Pillar 2: Universal UTC Timezone Normalization Engine

- **Implementation:** `src/normalization/utc.py`.
- **The Challenge:** Apache logs write `[14/Sep/2026:08:12:00 +0100]`, Linux `auth.log` writes `Sep 14 07:15:00` (no year, no offset), Windows writes ISO strings with localized time, and CloudTrail uses UTC `Z`.
- **The Solution:**
  1. Resolves missing years using ingestion-context anchors.
  2. Applies IANA timezone lookups (handling Daylight Saving Time transitions automatically via `zoneinfo`).
  3. Where an offset must be inferred from host metadata, Chronos permanently sets `offset_inferred = True`. This gives investigators transparency into which timestamps are absolute facts vs. forensic inferences.
  4. Identifies known machine clock skews (e.g. +47s drift on `WIN-ENG-07`) and flags them for timeline synchronization.

## Pillar 3: MITRE ATT&CK Technique-Level Tagging

- **Implementation:** `src/anomaly/tagger.py` and `config/attack_mappings.yaml`.
- **The Difference:** Many SIEMs only tag broad tactics (e.g., "Execution"). Chronos maps down to specific MITRE ATT&CK sub-techniques:
- `T1190` — Exploit Public-Facing Application ( `POST /login` command injection).
- `T1003.001` — OS Credential Dumping (Mimikatz execution / Event ID 4688 / LSASS access).
- `T1078` — Valid Accounts (Compromised engineer credentials).
- `T1021.004` — Remote Services: SSH (Lateral movement to Linux DB).
- `T1548.003` — Sudo and Sudo Caching (Root elevation to package databases).
- `T1560` — Archive Collected Data ( `tar` compression of proprietary aerospace schemas).
- `T1041` — Exfiltration Over C2/Cloud (AWS S3 `PutObject` into external bucket).

## Pillar 4: 4-Dimensional Cross-Host Correlation Engine

- **Implementation:** `src/correlation/engine.py`.
- **How It Works:** Rather than requiring an expensive graph database, Chronos implements a multi-hop entity join:
  1. **Hop 1 (External to Web):** External Attacker IP `198.51.100.42` hits `web-portal-01`.
  2. **Hop 2 (Web to Internal Pivot):** Firewall log matches outbound traffic from `web-portal-01` to internal engineer workstation `10.0.1.15` ( `WIN-ENG-07` ).
  3. **Hop 3 (Pivot to Credentials):** Workstation log records credential dumping for user `j.mitchell`.
  4. **Hop 4 (Credentials to Database):** Linux server `lin-db-03` logs an incoming SSH session from `10.0.1.15` authenticated as `j.mitchell`.
  5. **Hop 5 (Database to Cloud Exfiltration):** CloudTrail logs record credentials used to exfiltrate database archives to an external S3 bucket ( `s3-aerospace-vault` ).
- **Noise Filtration:** Out of 80 total events in the dataset, Chronos cleanly filters out 69 benign routine events (system crons, admin logins, health checks) and bundles the 11 malicious events into a single, cohesive attacker narrative.

## Pillar 5: Statistical Anomaly & Suspicious Time-Gap Detection

- **Implementation:** `src/anomaly/detector.py` .
- **Z-Score Spike Detection:** Evaluates event rates in tumbling 5-minute windows. If an event frequency exceeds the rolling mean by more than 2.5 standard deviations ( $Z > 2.5\sigma$ ), it is flagged as an execution spike. This automatically catches the 36-attempt SSH brute-force storm on `lin-db-03` ( $Z \approx 3.06\sigma$ ).
- **Dwell-Time Gap Detection:** Calculates the time delta between consecutive malicious actions. If a gap significantly exceeds expected automated tool latency (e.g., a 45-minute pause while the adversary analyzes stolen database tables before exfiltrating), it flags a "Suspicious Dwell Gap", pointing analysts directly to manual operator interaction.

---

## 4. Hackathon Live Demonstration Script (Step-by-Step)

Follow this scripted 3-to-4 minute walkthrough to present Chronos smoothly to hackathon judges.

### Preparation Checklist Before Presentation:

- Open Terminal 1 in project root: `d:\Hackathon\chronos\chronos`
- Open Terminal 2 in UI directory: `npm run dev` running at `http://localhost:5173`
- Have browser tab ready at `http://localhost:5173`
- Have code editor or terminal open to run Python scripts

---

### Step 1: The Hook & Introduction (30 seconds)

*"Hello judges! We present Chronos — an automated forensic timeline reconstruction engine for incident response.*

*In real-world cyber breaches, incident responders are handed gigabytes of messy, disconnected logs: Linux syslog, Windows EVTX, Apache web traffic, Cisco firewalls, and AWS CloudTrail. Clocks are skewed, timezones don't match, and finding the single attacker narrative among tens of thousands of benign events takes days. Chronos solves this end-to-end: from raw log intake and cryptographic hashing, to universal UTC normalization, ATT&CK correlation, statistical anomaly detection, and an interactive investigation dashboard."*

---

### Step 2: Live Pipeline Execution in the Terminal (45 seconds)

Run the pipeline live in front of the judges:

```
python scripts/run_pipeline.py
```

What to point out on screen as it runs:

1. **Speed:** The entire pipeline ingests, hashes, parses, normalizes, tags, correlates, and exports in under **0.15 seconds!**
2. **Stage 1 (Ingestion & Integrity):** Point to the terminal output:

"Notice how Stage 1 computes streaming SHA-256 hashes for all five raw sources before touching a single record, writing an immutable `ingest_manifest.json` for legal chain of custody."

3. **Stage 2 (UTC Normalization):** Point to the `Offset-inferred (flagged): 53` line:

"Notice here: Chronos automatically mapped all local timestamps to universal UTC, resolved missing syslog years, and explicitly flagged 53 timestamps where timezone offsets were inferred, ensuring total defensibility."

4. **Stage 3 & 4 (Correlation):** Point to:

"Out of 80 total heterogeneous events, Chronos filtered 69 benign background operations and isolated the exact 11 attacker events spanning across four separate machines."

5. **Stage 5 (Anomalies):** Point to:

"Our z-score detector caught the authentication storm on the database server with a Z-score of 3.06."

### Step 3: Interactive Dashboard Walkthrough (90 seconds)

Switch to the browser at `http://localhost:5173`:

#### A. Executive Summary & KPIs (Header & Cards)

- Point to the top KPI cards: **Attacker Events (11)**, **Hosts Compromised (4)**, **Techniques (7)**, and **Evidentiary Integrity (Verified)**.
- Point to the **Incident Narrative**: Show how Chronos auto-generated the chronological story:

"Here is the reconstructed incident narrative: Initial access via web injection, lateral movement to engineer workstation `WIN-ENG-07`, credential dumping with Mimikatz, SSH pivoting to database server `Lin-db-03`, and S3 cloud exfiltration."

#### B. Interactive Zoomable & Pannable Timeline (Click "Timeline" Tab)

- **Perform the Interaction:** Scroll your mouse wheel over the timeline to zoom in/out, and click-and-drag horizontally to pan across event clusters!

"Judges, this isn't a static chart. Responders can zoom deep into microsecond clusters of logs or zoom out to see the multi-day dwell time, with each event color-coded by MITRE ATT&CK tactic."

- Hover over an event node to show the tooltip with timestamps, hostnames, and technique IDs.

#### C. Cross-Host Lateral Movement Graph (Click "Host Graph" Tab)

- Show the visual topology:

"Here is our topological lateral movement graph. You can visually track the breach from the external internet edge ( `web-portal-01` ), jumping through the firewall ( `fw-edge-01` ) to workstation `WIN-ENG-07`, pivoting to `Lin-db-03`, and exfiltrating out to AWS S3."

#### D. MITRE ATT&CK Matrix (Click "ATT&CK Matrix" Tab)

- Show the tactic breakdown from Initial Access through Exfiltration with technique badges.

## E. Anomaly & Gap Inspector (Click "Anomalies" Tab)

- Point to the Z-Score Spike Detection card:

*"Here is our statistical anomaly detector. It flagged the SSH brute-force storm on `lin-db-03` (\$Z = 3.06\$) and highlighted the 42-minute dwell-time gap where the attacker manually staged database files."*

## F. Evidentiary Chain of Custody (Click "Custody" Tab)

- Show the raw source manifest:

*"For court admissibility, here is our Chain of Custody tab displaying the raw SHA-256 hashes of `auth.Log`, `access.Log`, `cisco.Log`, `windows_events.json`, and `cloudtrail.jsonl` with an automated verified status."*

## G. One-Click Forensic Export

- Click the **Export** button in the header to show instant download of `chronos-report.json`, and mention that CSV and executive text summaries are auto-saved to `data/exports/`.

---

## Step 4: Show Automated Test Suite (30 seconds)

Run in terminal:

```
python -m unittest discover tests
```

*"To ensure enterprise reliability, we have 20 automated unit and integration tests covering parser edge cases, leap years, timezone offsets, clock skew, and multi-hop correlation recall. All 20 tests pass cleanly in under 0.2 seconds."*

---

## Step 5: Conclusion & Vision (15 seconds)

*"In summary, Chronos turns hours of painful log analysis into a 150-millisecond automated forensic reconstruction with verified evidentiary integrity. Thank you, and we welcome your questions!"*

---

## 5. Tough Questions & Winning Answers for Judges (Q&A Defense)

Below are the exact technical questions experienced security, forensic, and cloud judges will ask, accompanied by winning answers:

## Q1: "How do you guarantee evidentiary integrity and chain of custody if you are modifying and parsing the logs?"

Winning Answer:

"We adhere strictly to ISO/IEC 27037 forensic standards: we never modify the raw evidence files. In `src/ingestion/manifest.py`, our first step before any parsing is computing a streaming SHA-256 cryptographic checksum of each raw log file and storing it in an immutable `ingest_manifest.json`. All normalizations, ATT&CK tags, and correlation links are stored in separate event schemas and our indexed database. Furthermore, we provide a built-in `verify_manifest()` function that can be executed at any time during a trial or audit to re-hash the source files on disk and mathematically prove they remain identical to their ingestion state."

## Q2: "What happens if a log has no year, no timezone offset, or the system clock had a 45-second drift?"

Winning Answer:

"That is one of the biggest pitfalls in real-world DFIR, and we solved it in `src/normalization/utc.py`:

1. Missing Years: Legacy syslog formats (e.g. `Sep 14 07:15:00`) omit the year. Chronos anchors the log to the ingestion period or configurable incident context.
2. Missing Timezone Offsets: If a host does not write its UTC offset, Chronos references host configuration metadata using Python's standard `zoneinfo` database (which correctly accounts for historical Daylight Saving Time shifts).
3. Defensibility Flag: Crucially, whenever Chronos infers an offset, it marks `offset_inferred = True` on the event. An analyst or attorney can immediately filter between explicit log facts and inferred values.
4. Clock Skew: If a forensic host inspection reveals an unsynchronized RTC (like the +47-second clock drift on `WIN-ENG-07` in our test scenario), Chronos accounts for this delta during correlation so events across systems line up chronologically."

## Q3: "How does your correlation algorithm scale beyond 80 events to 500,000 or millions of logs?"

Winning Answer:

"Our architecture was designed specifically for high throughput:

1. Streaming Ingestion: All five of our parsers are written as iterative Python generators ( `yield ChronosEvent` ). We never load entire multi-gigabyte log files into memory at once.
2. Time-Indexed Inverted Index: Events are persisted in an indexed store where `utc_epoch_ns` is indexed.
3. Sliding-Window Entity Joins: Our correlation engine in `src/correlation/engine.py` doesn't perform an  $O(N^2)$  all-pairs comparison. It partitions events into configurable sliding time windows (e.g., 60 minutes) and indexes on four primary keys: `src_ip` , `user` , `session_id` , and `host` .
4. Elasticsearch-Ready: We defined a `StorageBackend` abstract base class in `src/storage/base.py` . While our offline demo uses SQLite, swapping to an Elasticsearch or ClickHouse bulk backend requires changing only one file."

#### Q4: "Why use rule-based ATT&CK tagging and z-score anomaly detection instead of an LLM or deep learning?"

Winning Answer:

"In digital forensics and incident response, explainability and determinism are non-negotiable.

1. If an analyst presents evidence to executive leadership or a court of law, saying 'our LLM classified this' is unacceptable due to hallucinations, non-deterministic outputs, and lack of auditability.
2. By using data-driven YAML rule definitions in `config/attack_mappings.yaml` , our technique tags are 100% deterministic, inspectable, and instantly updatable by SOC teams as new CVEs emerge.
3. Similarly, for anomaly detection, rolling z-score calculations ( $Z > 2.5$ ) provide a transparent, mathematically proven baseline for identifying volume spikes without training black-box neural networks that produce unpredictable false positives."

#### Q5: "How does Chronos separate benign background noise from the actual attack?"

Winning Answer:

"Through multi-hop entity linking and confidence scoring:

In our synthetic aerospace dataset, there are 69 benign events: routine admin SSH logins, periodic cron jobs, and normal web browsing.

Chronos does not flag an event simply because it's an SSH login or a web hit. An event enters the Attacker Narrative only if it chains across entities to an antecedent suspicious event:

- The web hit on `/login` had an exploit payload ( `T1190` ).
- The external IP that triggered the exploit matches the firewall teardown connection to `10.0.1.15` .
- The user account `j.mitchell` that logged in via SSH originated directly from that pivot host.

Standalone benign events lack these chained linkages and are kept in the event table for context but excluded from the attack narrative."

## Q6: "Can this be deployed in an air-gapped or classified environment?"

Winning Answer:

"Yes, 100%. Chronos has zero external cloud API dependencies:

- Python runs purely standard library and local packages ( `zoneinfo` , `sqlite3` , `hashlib` ).
- The React frontend compiles to static HTML/JS/CSS that can be hosted locally or bundled in an offline desktop container like Electron or Tauri.
- No telemetry, no external AI calls, no internet connection required. It is ready for air-gapped SOCs and secure SCIF environments today."

## Q7: "What are the limitations today and what is your future roadmap?"

Winning Answer:

"Today, Chronos is an MVP focused on post-incident forensic ingestion and reconstruction. Our immediate next milestones are:

1. Direct Binary EVTX Parsing: Integrating `python-evtx` for raw `.evtx` binary chunks alongside our current JSON/XML parser.
2. Real-time Live Tailing: Adding a Kafka or syslog-ng listener to tail logs in real-time rather than purely batch files.
3. Sigma / YARA-L Integration: Allowing SOC engineers to import standard Sigma detection rules directly into our ATT&CK tagging engine.
4. Automated PDF Executive Report: Hooking our export engine directly into ReportLab to output sealed PDF court summaries with digital signatures."

## 6. Key Metrics, Data Flow & Evidence Summary

### By the Numbers (Verification Data)

| Metric                       | Chronos Result              | Forensic Implication                            |
|------------------------------|-----------------------------|-------------------------------------------------|
| Raw Sources Parsed           | 5 distinct formats          | Linux, Windows, Apache, Cisco, AWS              |
| Pipeline Processing Time     | < 0.15 seconds              | Instantaneous triage for rapid IR               |
| Raw Events Ingested          | 80 events                   | Realistic sample including noise                |
| Benign Events Filtered       | 69 events                   | No false-alarm inundation for analysts          |
| Attacker Narrative Events    | 11 events                   | 100% ground-truth precision & recall            |
| Compromised Hosts Identified | 4 hosts                     | web-portal-01 , WIN-ENG-07 , lin-db-03 , s3     |
| ATT&CK Techniques Tagged     | 7 techniques                | T1190, T1003, T1078, T1021, T1548, T1560, T1041 |
| Statistical Anomalies        | 1 spike (\$Z=3.06\$), 1 gap | Brute-force storm + 42m staging pause           |
| Evidentiary Integrity        | 100% match (0 mismatches)   | SHA-256 pre-parse verification                  |
| Automated Tests              | 20 / 20 passing             | Unit & E2E integration verification             |

### File Navigation Cheatsheet for Q&A:

If a judge asks to see the code for any specific feature:

| Judge Asks About...                      | Open This File                                                                                                                 |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| SHA-256 Ingestion & Chain of Custody     | <code>src/ingestion/manifest.py</code>                                                                                         |
| Timezone Normalization & Inferred Flags  | <code>src/normalization/utc.py</code>                                                                                          |
| Cross-Host Correlation & Entity Joins    | <code>src/correlation/engine.py</code>                                                                                         |
| MITRE ATT&CK Tagging Rules               | <code>src/anomaly/tagger.py</code> & <code>config/attack_mappings.yaml</code>                                                  |
| Statistical Spikes (Z-Score) & Time Gaps | <code>src/anomaly/detector.py</code>                                                                                           |
| Ingest & Pipeline Runner                 | <code>scripts/run_pipeline.py</code>                                                                                           |
| Interactive Zoomable Timeline Component  | <code>ui/src/components/chronos/attack-timeline.tsx</code>                                                                     |
| Topological Host Graph Component         | <code>ui/src/components/chronos/host-graph.tsx</code>                                                                          |
| Automated Test Suite                     | <code>tests/test_parsers_and_norm.py</code> , <code>tests/test_correlation.py</code> , <code>tests/test_e2e_pipeline.py</code> |